WE'RE HIRING

Senior Backend Developer

Lead the technical execution of AI-powered products. Architect scalable systems. Work directly with international clients.

REMOTE - FULL-TIME - 4+ YEARS

BDT 80,000 - 1,00,000 + free Claude Pro subscription

This document contains:

Part 1 The role, requirements, and how to apply

Part 2 The take-home task (please complete and submit with your application)

PART 1

The Role

CodeFusion AI is hiring a senior backend developer to lead technical execution on our AI-powered products. You'll own projects end-to-end, architect scalable systems, mentor junior engineers, and work directly with international clients.

This role is a good fit if you've spent the last few years building real backend systems in Python, have shipped production AI integrations, and are comfortable making architecture decisions and leading a small team, not just executing tickets.

Role Details

Position	Senior Backend Developer
Location	Fully remote
Type	Full-time
Working Hours	11:00 AM - 8:00 PM (Bangladesh Standard Time)
Experience	4+ years
Salary	BDT 80,000 - 1,00,000 (based on experience), plus a free Claude Pro subscription
Website	www.codefusionai.com

What You'll Do

- Lead the architecture and development of backend systems powering AI applications
- Design and maintain high-performance REST APIs in Python (Django, FastAPI)
- Own projects from planning through deployment and production support
- Mentor junior developers through code reviews, pairing, and technical guidance
- Manage development workflows: task assignment, sprint planning, delivery tracking
- Work directly with international clients to scope requirements and report progress
- Make architecture and system design decisions for SaaS and AI-driven platforms
- Build and maintain CI/CD pipelines, deployment workflows, and cloud infrastructure
- Collaborate closely with frontend, AI, and DevOps engineers on cross-functional delivery
- Use AI-assisted development tools (Claude Code, GitHub Copilot, Cursor) effectively as part of your daily engineering workflow

Required Skills

- 4+ years of professional backend development experience
- Strong Python skills with Django and FastAPI in production

- Solid grasp of system design, scalable architecture, and engineering fundamentals
- Production experience with PostgreSQL, MongoDB, or similar databases
- Hands-on experience with Redis, Celery, or comparable queue/caching systems
- Experience integrating LLM APIs and working with vector databases
- Experience building AI agents or agentic workflows (tool use, multi-step reasoning, orchestration frameworks like LangGraph, CrewAI, or similar)
- Strong understanding of authentication, API security, and background task processing
- Comfortable with Docker, CI/CD, and cloud deployment (AWS, GCP, Azure, or DigitalOcean)
- Demonstrated experience using Claude Code or equivalent AI coding agents as part of a real engineering workflow, including prompting strategies, agentic task delegation, code review with AI assistance, and knowing when *not* to use it
- Strong written and spoken English
- Experience leading or mentoring developers, ideally in a remote environment

Nice to Have

- Kubernetes and Terraform experience
- MLOps and AI deployment pipeline experience
- Familiarity with observability tools (e.g. Grafana, Datadog, Sentry)
- WebSockets and event-driven architecture in production
- Working knowledge of React.js or Next.js for cross-team collaboration
- Direct experience with international clients

Why CodeFusion AI

- Fully remote with flexible hours
- Real ownership over technical decisions, not just execution
- Direct work with international clients on AI-focused products
- Small, collaborative engineering team where your input shapes the roadmap
- Budget and encouragement for using modern AI development tools, including a **free Claude Pro subscription** provided by the company

How to Apply

Submit everything below in a single email to **hr@codefusionai.com** within **5 days** of receiving this document:

- CV
- GitHub profile or portfolio
- A short note on why this role interests you, and a brief example of how you've used Claude Code or a similar AI coding tool on a real project
- Your **completed take-home task** — see Part 2 for full details (public GitHub repo link, DECISIONS.md inside the repo, and video walkthrough link)

Subject line: Senior Backend Developer Application - [Your Name]

Next Step

The Take-Home Task

Part 2 is a take-home task you should complete and submit alongside your application materials.

It is designed to take roughly one weekend (8-12 hours of focused work) and evaluates the exact skills described in Part 1: AI agent development, backend architecture with Django, database modeling, REST API design, and effective use of AI coding tools.

You will have 5 days from receiving this document to submit your application, CV, and completed task together. See Part 2 for full details.

PART 2

Take-Home Task

Overview

Build a small but complete AI agent that solves a real backend problem. This task is designed to take roughly a weekend (approximately 8-12 hours of focused work). We're more interested in your design decisions, code quality, and judgment than in scope or polish, please don't over-invest your time.

You will have **5 days** from receiving this document to submit your application together with the completed task.

The Task: Build a Codebase Research Agent

Build an AI agent that can answer technical questions about a given GitHub repository by exploring the code itself.

Functional Requirements

The agent should accept:

- A public GitHub repository URL (or local path)
- A natural language question about the codebase

And produce:

- A clear, accurate answer
- References to specific files, functions, or line numbers it used to form the answer

Every research session must be **persisted to the database** so it can be retrieved, reviewed, and built upon later. The agent itself should be able to read from and write to the database as part of its workflow.

Example Inputs and Expected Behavior

Input: `https://github.com/tiangolo/fastapi` - "How does FastAPI handle dependency injection internally?"

Expected: The agent explores relevant files, identifies the dependency injection mechanism, and explains it with references to specific files and code locations.

Input: `https://github.com/celery/celery` - "Where is task retry logic implemented, and what backoff strategies are supported?"

Expected: A focused answer pointing to the relevant module(s), with a brief explanation of retry behavior.

Technical Requirements

- **Language:** Python
- **Framework:** Django (required)

- **Database:** PostgreSQL (preferred) or SQLite for simplicity. Use Django ORM, no raw SQL unless justified.
- **LLM:** Your choice (Claude, GPT-4, open-source, whatever you prefer). Document why you chose it.
- **Agent design:** The agent must use **tool calling / function calling** to explore the repository AND interact with the database. Examples:

  Code exploration tools: `list_files(path)`, `read_file(path)`, `search_code(query)`, `get_file_summary(path)`

  Database tools: `save_finding(session_id, file_path, note)`, `get_previous_findings(repo_url)`, `list_past_sessions(repo_url)`
- **Multi-step reasoning:** The agent should be capable of making multiple tool calls before producing a final answer
- **Interface:** A REST API (Django views or Django REST Framework). No CLI, no frontend needed. At minimum, expose endpoints to start a research session, retrieve results, and list past sessions for a repo.

Database Requirements

Design and implement a schema that supports the agent's workflow. At minimum, you should model:

- **Repository**: repos that have been researched (URL, name, last analyzed timestamp, etc.)
- **ResearchSession**: each question asked against a repo (question text, final answer, timestamps, token usage if you track it)
- **Finding / ToolCall log**: what the agent looked at and what it concluded along the way

You're free to design the schema as you see fit, we want to see your modeling choices. Use Django migrations.

The agent should:

- **Write** to the database during a research session (logging what it found, what tools it called, the final answer)
- **Read** from the database when relevant, for example, if the same repo was researched before, the agent should be able to check prior findings before re-exploring from scratch
- Be invokable in a way that creates a new record and links related findings to it

We expect to see at least a few sample records created by actually running the agent, include a fixture, seed script, or instructions for generating sample data.

What We're NOT Looking For

- Production-grade error handling on every edge case
- A polished UI
- Authentication, multi-user support, or scaling
- Comprehensive test coverage (a few key tests are enough)

What We ARE Looking For

- **Agent design decisions:** How did you structure the tool set? When does the agent stop? How do you prevent infinite loops?
- **Database design:** Is your schema sensible, normalized where it should be, denormalized where it makes sense?
- **Agent-DB interaction:** Does the agent meaningfully use the database, or is persistence just an afterthought tacked on?
- **Context management:** How do you handle large codebases that exceed the context window?
- **Cost and latency awareness:** Did you think about token usage and number of LLM calls?
- **Code quality:** Clean, readable, well-organized Python and Django code
- **Judgment:** What did you choose to build well vs. leave rough? Why?

Deliverables

Submit all three of the following:

1. GitHub Repository

- Public repository
- Clear `README.md` with setup and run instructions
- Working code that runs from a clean clone with documented steps
- Include a sample `.env.example` for any API keys required (do NOT commit real keys)

2. Written Explanation (DECISIONS.md in the repo)

A short document (target: 500-800 words) covering:

- Architecture overview, how the agent is structured
- **Database schema rationale**: why you modeled it this way, what trade-offs you considered, what you'd change at scale
- Key design decisions and trade-offs you made
- What you'd do differently with more time
- How you used AI coding tools (Claude Code, Cursor, Copilot, etc.) during this task, what worked, what didn't, what you did manually
- Any limitations or known issues

3. Short Video Walkthrough (5-10 minutes)

The video must be in English. This role involves direct communication with international clients, so spoken English fluency is part of what we evaluate.

A screen recording (Loom, YouTube unlisted, or similar) showing:

- A live demo of the agent answering 1-2 questions on a real repo, via API calls (Postman, curl, or HTTPie are all fine)
- A quick walkthrough of the code structure and database schema

- Sample records in the database after the agent has run (Django admin, `dbshell`, or any DB browser is fine)
- Your explanation of the most interesting design decision you made
- **If you used AI coding tools (Claude Code, Cursor, Copilot, ChatGPT, or similar), walk us through your complete AI usage process.** Cover: which tool(s) you used and why, how you prompted them (general approach, not every prompt), which parts of the codebase were AI-generated vs. hand-written vs. AI-assisted then edited, how you reviewed and verified AI output, where AI helped most, and where it failed or led you astray. We want to see your real workflow, not a sanitized version. Honesty here is a strong signal.

Evaluation Criteria

We'll evaluate submissions on:

Area	What we're looking for
Agent design	Sensible tool design, reasonable stopping conditions, multi-step reasoning that actually works
Database modeling	Clear, justifiable schema; sensible relationships; meaningful use of the DB by the agent (not just a log dump)
Code quality	Readable, organized, idiomatic Python and Django
Engineering judgment	Right level of polish, sensible trade-offs
Communication	Clear README, thoughtful DECISIONS.md, coherent walkthrough
AI tool usage	Effective use of AI coding tools without losing engineering ownership

Ground Rules

- **Use any AI tools you want.** This role requires daily AI-assisted development, so we expect you to use Claude Code, Cursor, Copilot, ChatGPT, or whatever fits your workflow. We want to see how you work *with* these tools, not without them.
- **Be honest about what's yours and what's AI-generated.** We're not testing whether you can write code without AI, we're testing whether you can ship good systems using it.
- **No need to over-engineer.** A small project done well beats a large project done poorly.
- **Ask questions.** If something is genuinely unclear, email us. We won't penalize good questions.

Submission

Include your GitHub repo link and video link with your application email (see **How to Apply** in Part 1). Send everything together in a single email to **hr@codefusionai.com** with the subject line:

Senior Backend Developer Application - [Your Name]

We aim to review submissions and respond within 5 business days.

If you have a portfolio piece you've already built that demonstrates similar skills (a production AI agent you've shipped), feel free to submit that instead with an explanation, we value your time and don't want to duplicate work you've already done.